

VIT BHOPAL UNIVERSITY
BUILD YOUR OWN PROJECT

SMART STUDENT ACADEMIC MANAGEMENT SYSTEM

Web-Based Academic Management Application

Submitted By	Shriya Parmar
Registration Number	25BAI10307
Program	B.Tech – Artificial Intelligence & Machine Learning
Academic Year	2026–2027

Project Report

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Functional Requirements
6. Non-Functional Requirements
7. System Modules
8. Technology Stack
9. System Architecture
10. UML and Design Views
11. Implementation
12. Application Screenshots
13. Testing
14. Challenges Faced
15. Learnings
16. Future Enhancements
17. Conclusion
18. References

1. Introduction

The Smart Student Academic Management System is a web-based application developed to simplify the management of student information and academic records. It provides a centralized platform for managing student details, subjects, marks, attendance and performance.

The application is implemented using Java, Spring Boot, Spring MVC and Thymeleaf. HTML and CSS provide the interface, while Java serialization provides persistent file-based storage.

2. Problem Statement

Managing student academic information manually can be time-consuming and difficult to organize. Student details, marks, attendance and performance records may be maintained separately, making analysis and report generation inefficient.

The proposed system provides a centralized web-based solution for storing, validating and analyzing academic information.

3. Objectives

- Manage student information efficiently.
- Maintain academic records including subjects, credits, marks and attendance.
- Calculate grades, average marks and GPA automatically.
- Identify weak subjects and attendance warnings.
- Generate complete student academic reports.
- Validate input and provide persistent storage.

4. Scope of the Project

- Student CRUD operations.
- Academic record management.
- Performance analytics.
- Weak-subject and attendance warnings.
- Report generation.
- Java serialization-based persistence.
- Browser interaction through Spring Boot and Thymeleaf.

5. Functional Requirements

ID	Requirement	Description
FR1	Student Management	Add, search, update, display and delete student records.
FR2	Academic Management	Add subject, credits, marks and attendance records.
FR3	Grade Calculation	Calculate grade and grade point from marks.
FR4	Performance Analytics	Calculate average, GPA and performance status.
FR5	Warnings	Identify weak subjects and attendance below the defined threshold.
FR6	Reporting	Generate and view an academic report for a student.
FR7	Validation	Validate student data, marks, credits and attendance.

ID	Requirement	Description
FR8	Persistence	Save and load student and academic records from files.

6. Non-Functional Requirements

- Usability: Simple browser-based pages, clear forms and navigation.
- Reliability: Validation and exception handling reduce invalid data and application errors.
- Maintainability: Separate model, service, controller and exception packages.
- Performance: Lightweight local file storage and direct calculations provide responsive operations.
- Data Persistence: Student and academic records are stored in serialized files.
- Error Handling: Invalid marks, attendance, credits and student details are rejected with messages.
- Scalability: The modular structure allows database storage and authentication to be added later.
- Security: Input validation and controlled server-side processing are used; future versions can add authentication and authorization.

7. System Modules

- 1. Student Management** – Manages student registration number, name, email, program and semester. Supports add, search, update, display and delete operations.
- 2. Academic Management** – Stores subject code, subject name, credits, marks and attendance for each student.
- 3. Performance Analytics** – Calculates average marks, GPA, grades, performance status, weak subjects and attendance warnings.
- 4. Student Reports** – Combines student details and academic records into a complete academic report.
- 5. Validation & Persistence** – Validates input and stores data using Java serialization through dataManager.

8. Technology Stack

- Java 21: Core programming language and object-oriented implementation.
- Spring Boot: Application framework and runtime.
- Spring MVC: Controller-based web request handling.
- Thymeleaf: Server-side HTML template rendering.
- HTML5 / CSS3: Web page structure and styling.
- Maven: Project build and dependency management through Maven Wrapper.
- VS Code: Development environment.
- Java Serialization: Persistent storage of students.dat and academic_records.dat.

9. System Architecture

The system follows a layered architecture. Users interact through a browser; Spring Boot controllers receive HTTP requests and connect the web layer with service and model classes. dataManager performs serialization and persistent file storage.

System Architecture Diagram

Smart Student Academic Management System

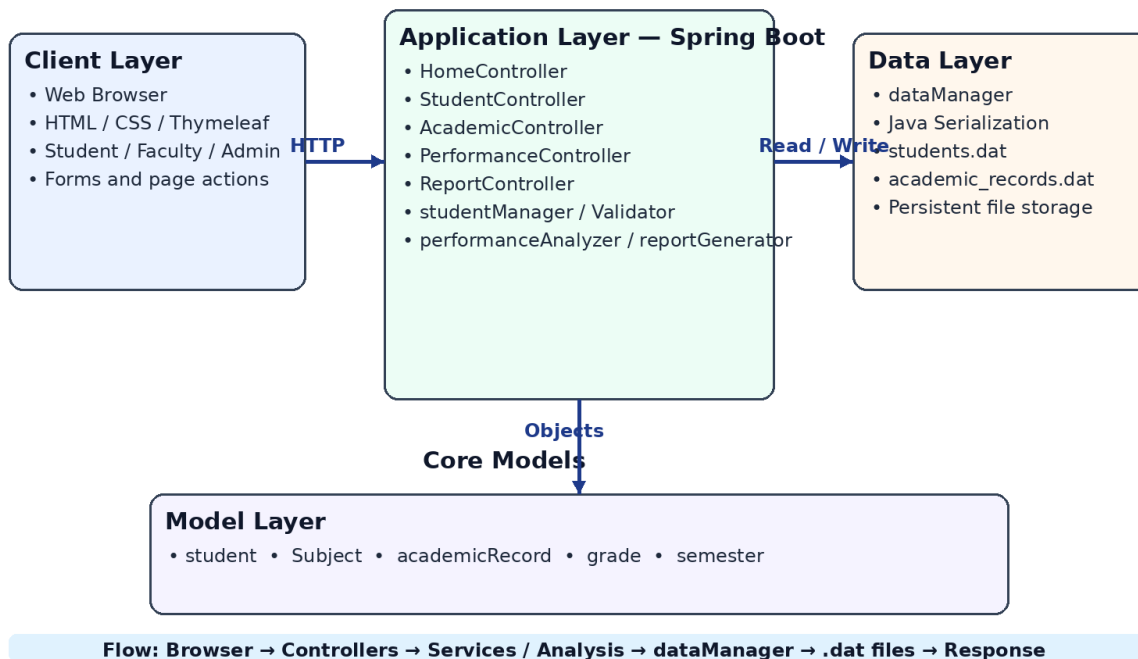


Figure 6. System Architecture of the Smart Student Academic Management System

Main flow: Browser → Controller → Service / Analysis → Model and dataManager → Persistent files → Response.

10. UML and Design Views

10.1 Use Case View

- Student / User: Open dashboard, manage student records, add academic records, view performance and view reports.
- Faculty / Academic User: Maintain student academic data, inspect marks and attendance, analyze performance and reports.
- System: Validate inputs, calculate grades/GPA, store data and generate reports.

10.2 Class / Component View

- student: Stores registration number, name, email, program and semester.
- Subject: Stores subject code, subject name and credits.
- academicRecord: Associates a Subject with marks and attendance.
- grade: Provides static grade and grade-point calculations.
- studentManager: Handles student CRUD operations and persistence.
- studentValidator: Validates student and academic input.
- performanceAnalyzer: Calculates average, GPA, performance status, weak subjects and attendance warnings.
- reportGenerator: Generates a formatted student academic report.
- dataManager: Serializes and deserializes student and academic records.
- Controllers: Connect browser requests to the application logic and templates.

10.3 Sequence / Workflow View

User → Browser → Controller → Service/Manager → Model/dataManager → File System → Controller → Browser

10.4 Data Flow

students.dat stores student objects and academic_records.dat stores the academic record map. Java serialization is used for object persistence.

11. Implementation

The existing Java academic management logic was integrated into a Spring Boot web application. Model and service classes are organized under src.model, src.service and src.exception, while web controllers are under com.student.student_management.

- HomeController – dashboard and student count.
- StudentController – student CRUD pages.
- AcademicController – academic record creation and validation.
- PerformanceController – performance data preparation.
- ReportController – student report preparation.
- studentManager – student CRUD and persistence.
- performanceAnalyzer – average, GPA and warning analysis.
- dataManager – serialized file storage.

11.1 Validation and Error Handling

Server-side validation checks required fields, semester range, credits, marks and attendance. Invalid values are rejected and returned to the form with an error message.

11.2 Data Persistence

Student objects are stored in students.dat and academic records in academic_records.dat using ObjectOutputStream and ObjectInputStream.

12. Application Screenshots

The following screenshots were captured from the running Spring Boot application at localhost:8080.

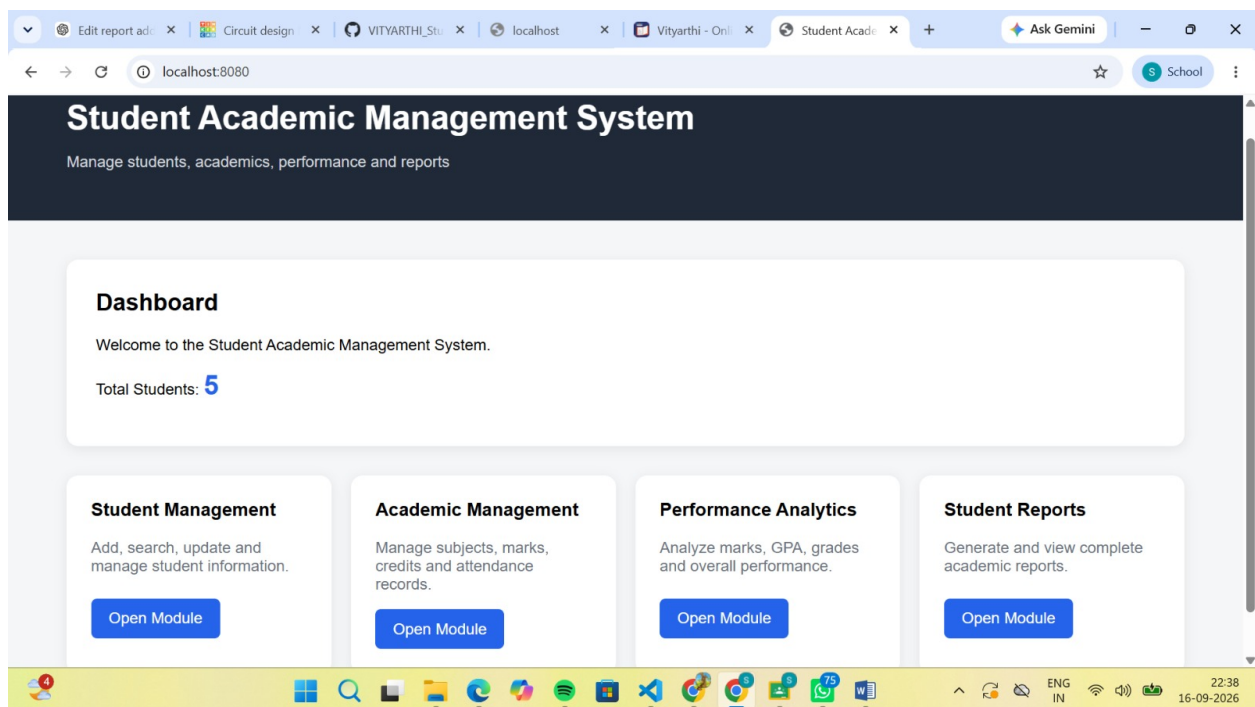


Figure 1. Application Dashboard

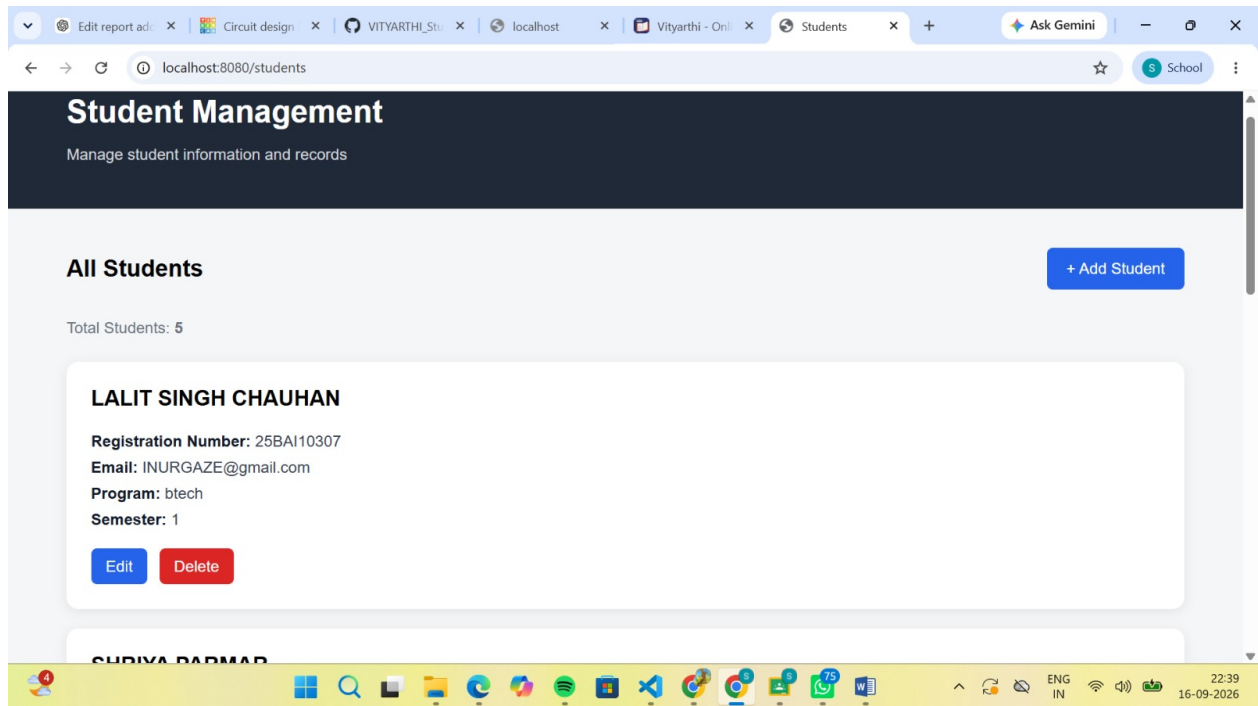


Figure 2. Student Management Module

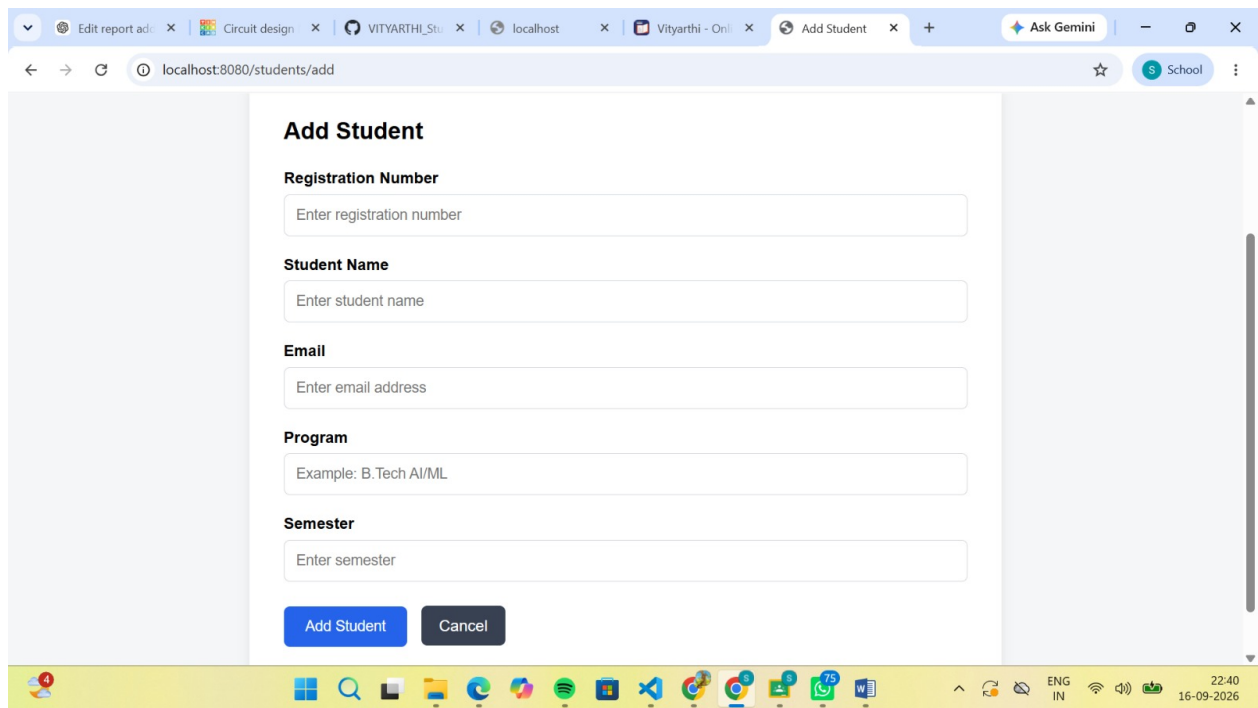


Figure 3. Add Student Form

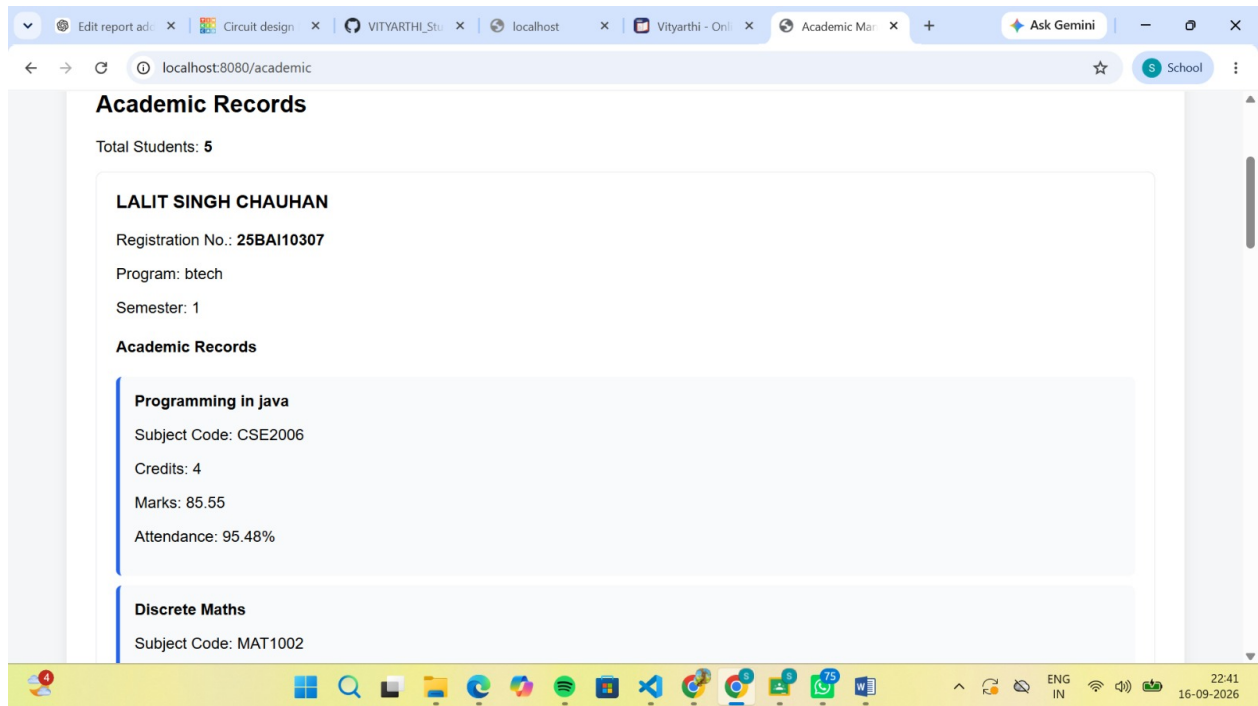


Figure 4. Academic Records Module

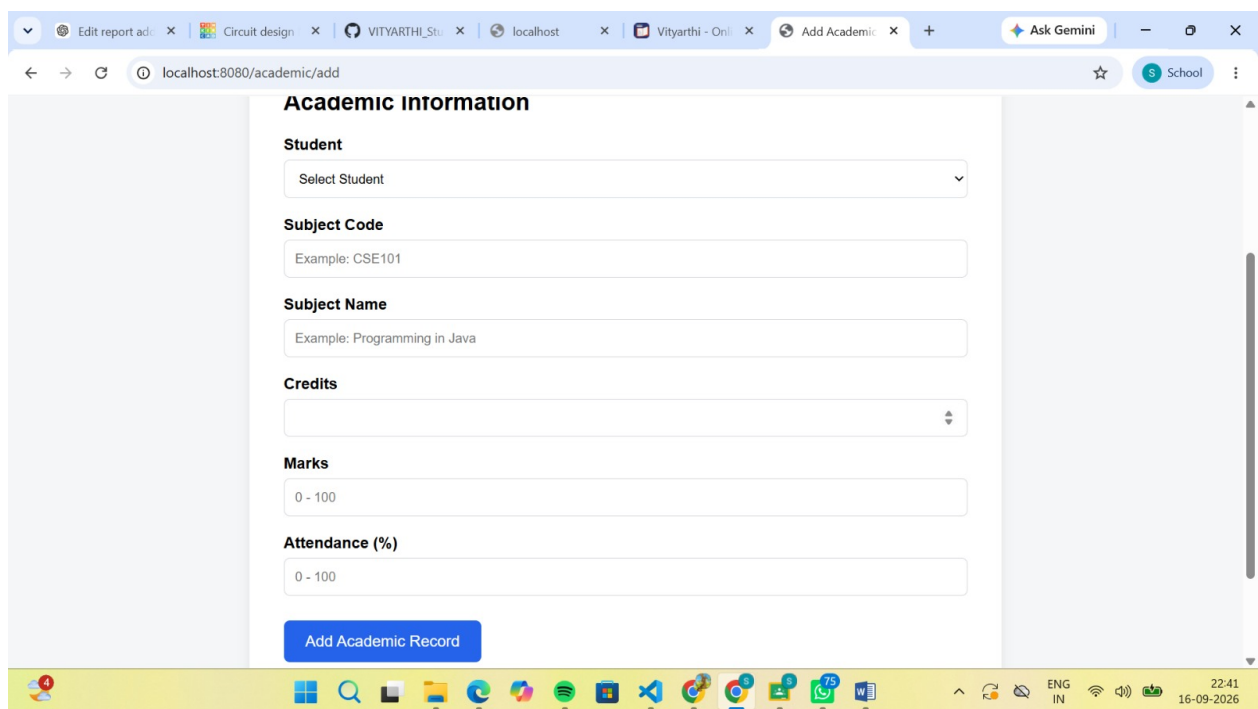


Figure 5. Add Academic Record Form

13. Testing

Test Case	Input / Action	Expected Result	Status
Add Student	Valid student details	Student added successfully	Passed
Edit Student	Updated student details	Student information updated	Passed
Delete Student	Registration number	Student removed	Passed
Search Student	Registration number	Matching student displayed	Passed
Add Academic Record	Valid marks and attendance	Record saved successfully	Passed

Test Case	Input / Action	Expected Result	Status
Invalid Marks	Marks > 100	Validation error displayed	Passed
Invalid Attendance	Attendance > 100	Validation error displayed	Passed
Performance Analysis	Student academic records	GPA, grades and status displayed	Passed
Student Report	Select student	Complete academic report generated	Passed
Data Persistence	Restart application	Saved records remain available	Passed

14. Challenges Faced

- Converting console functionality into Spring Boot web pages.
- Managing package names and lowercase student class naming.
- Running with Maven Wrapper because Maven was not globally available.
- Connecting Thymeleaf forms with Spring MVC controllers.
- Loading current data correctly across pages.
- Implementing server-side validation and error messages.
- Maintaining persistent serialized records.

15. Learnings

- Spring Boot project structure and Maven execution.
- HTTP GET/POST request handling through controllers.
- Thymeleaf model-to-HTML rendering.
- Object-oriented design using model and service classes.
- ArrayList and HashMap based record management.
- Exception handling and validation.
- Java object serialization.
- Conversion of console applications into web applications.

16. Future Enhancements

- MySQL/database integration.
- Secure login and role-based access.
- Advanced search and filtering.
- Semester-wise and subject-wise analytics.
- PDF report export.
- Automated unit and integration testing.
- Cloud deployment.
- Backup and restore functionality.

17. Conclusion

The Smart Student Academic Management System provides a centralized web-based solution for managing student and academic information. It combines CRUD operations, academic record management, validation, automated grade and GPA calculation, performance analysis and report generation. The project demonstrates practical use of Java, object-oriented programming, Spring Boot, Spring MVC, Thymeleaf, file handling, serialization and exception

handling.

18. References

- VITYarthi – Build Your Own Project guidelines provided for the project.
- Spring Boot documentation and Spring MVC concepts.
- Java Platform documentation for collections, exception handling and object serialization.
- Thymeleaf documentation for server-side HTML templates.
- Apache Maven documentation and Maven Wrapper usage.

GitHub Repository: VITYARTHI_Student-Academic-Management-System_JAVA